AS OF 1 NOVEMBER 2026 · WRITTEN BY JILL MORENO

Designer Workflow on the Hub

PART 0

Before you start

This guide is the day-to-day: sit down, build something, get it in front of the team. It assumes your machine is already set up.

> **First time on this project?** Go to First Time - Contributing to the Hub.

Most Important Thing.
You work on YOUR BRANCH. Never push directly to main.
If you ever find yourself on main, stop and tell Jill.

PART 1

Every time you sit down

Open Claude → the Code tab → the `xcel-ux-dashboard` folder. Start a new conversation for each piece of work.

How to talk to it

1. **Plain sentences, one step at a time.** One request to a message, not three:

   YOU: Rebase my branch onto main.

2. **Read the line before, and the summary after.** Claude says what it is about to do, then what it did. If either surprises you, ask before going on.

3. **Allow the permission card when it matches what you asked for.** If it describes something else, click Deny and ask what it was trying to do.

4 **A refusal naming a file is correct.** That file is the Hub's own machinery and only Jill changes it. Tell her what you needed.

5 **Type `/` for the routines.** Yours are `/share-component`, `/share-branch` and `/share-to-stage` — Part 9 says what each one does.

Then ask one question

YOU

> Which branch am I on, do I have uncommitted changes, and how far behind main am I?

Your own `feat/…` branch, changes you recognise, under ten commits behind — go. Anything else:

- On `main` — stop and tell Jill.
- Changes you do not recognise — ask Claude what they are before touching anything.
- Ten or more behind — rebase before you build (Part 2).

Rebasing every few days is painless. Rebasing once at the end of a month is not.

PART 2

Keeping your branch current

Main moves under you — several times a week at the moment. A rebase replays your commits on top of today's main, as if you had started this morning. It does not change your work.

Rebase every few days.

Three days behind is a minute's work; a month behind is an afternoon.

Most days it is one sentence

YOU

> Commit anything uncommitted, make a backup branch, then fetch origin and rebase this branch onto origin/main. Run the build and the tests, then push with force-with-lease. If anything conflicts, stop and show me both sides.

Allow it. The backup means nothing that follows can lose work — you can delete it next week. Expect it to stop if main has edited the same files you have: that is the normal case when two people work on one surface, and the next heading is what to do about it.

Commits of yours that already reached main in another form — through someone else's promotion, say — are recognised and dropped on the way. Coming out with fewer commits than you went in with is normal, not lost work.

If it stops on a conflict

Claude names the file and shows two versions of the same lines: what main has, and what your commit has. Read both, then say one of:

YOU

> Keep my version. / Keep main's version. / Keep both — main's block first, then mine.

YOU

> Continue the rebase.

It may stop again on the next commit; do the same thing. If you cannot tell which to keep, say **"Abort the rebase"** — your branch goes back exactly as it was — and ask Jill.

Then look at it

YOU

> How far behind main am I now, and what is the branch link?

You want **0 behind**. Netlify takes two to four minutes to build, and the deploys page shows where it is up to. Click through from the page a reviewer would start on: if it looks like it did before the rebase, you are done. If something that used to work is broken, that is where main and your branch disagree — tell Jill which page.

> Or have Claude do all of this for you every weekday morning — Part 10.

PART 3

Building a change, start to finish

This is the loop you will repeat for every piece of work. Each step is one sentence to Claude.

1 **Start from main.**

YOU

> Fetch origin, then make a new branch called feat/short-name from origin/main and switch to it.

Never branch from an old branch of yours; it drags the old branch's history along.

2 **Say what you want.**

Describe the change as you would to a developer sitting next to you. Name the page, the component, what it should do instead. Claude reads the code and proposes where the change goes. If it names a file you don't recognise, ask "What is that file?" before saying yes.

3 **Changing something that already exists? Use a sibling copy.**

YOU

> /share-component

Name the component. It copies `StudyPaceTile.tsx` to `StudyPaceTileV2.tsx`, leaves the original untouched, and adds a flag that picks between them. This is why your branch almost never conflicts with anyone else's: nobody edits the same file.

4 **The flag is on by default on your branch.**

Claude does this when you say "behind a feature flag that is on by default on this branch". It means your branch link shows your version, and Jill can decide at merge whether it ships. Ask "What is the flag called?" and write it down; the branch link can turn it off with `&ff=flag-name:off` if you want to show the before.

5 **Check it.**

YOU

> Run the type check, the tests, and lint on the files I changed.

If you added or edited anything under `public/prototypes/`, add "and the smoke suites". Red means fix before pushing: "Fix that and run it again."

6 **Commit.**

YOU

> Commit everything on this branch with a message that says what changed and why.

Claude writes the message in the project's style and shows it to you first.

7 **Push.**

YOU

> Push my branch to GitHub.

The first push on a new branch also sets it up on GitHub; Claude handles that. From then on, every push is the same sentence.

8 **Wait for Netlify.**

Two to four minutes. If you want to know rather than guess:

YOU

> Check whether my branch has finished building on Netlify.

Or watch it yourself: the deploys page lists every branch link, newest first, with Building, Published or Failed on each. Find yours by branch name. You need to be signed in to the Netlify project — ask Jill if you are not on it.

9 **Get the link.**

YOU

> Give me the branch link for this branch.

The shape is `https://‹branch-with-dashes›--ux-design-xceldashboard.netlify.app` followed by the page: `/dashboard-rebrand?demo=1` for the dashboard, `/dashboard-rebrand?demo=1&section=compass` for Compass, `/end-to-end-demo?demo=1` for the invite flow. `?demo=1` turns on the demo controls bar.

10 **Look at it the way a reviewer would.**

Click through from the page they would start on, not straight to your component. Three bugs in September were invisible on a direct link and obvious on the first click.

11 **Keep going.**

Each further change is step 2, then `/share-branch` again. Rebase every few days (Part 2) so the final merge is boring. When the next change is really a different piece of work, start a new branch for it rather than letting this one grow.

> **Steps 5 to 9 are one command.** `/share-branch` runs the checks, commits, pushes, waits for the build and hands you the link, then opens the Refinement form — Part 4. The long way above is what it is doing, which is worth knowing for the day it stops halfway.

Putting a decision on the design bar

A prototype carries two control bars, and only one of them is yours.

BAR	SHOWS	WHOSE
Demo controls — amber	How the product behaves for different learners: persona, progress, tier, education.	Jill's. Its control list says what the *product* demos, not what your exploration is exploring.
Design controls — light cyan	The decisions still open inside your own exploration: fonts, brand colour, layout axes.	Yours, scoped to the dashboard version you are working on.

You never edit either bar to get a control onto it. Both files are protected and Claude will refuse. You mark the flag you already own instead:

YOU

Make my flag a design control so it shows on the design bar.

That writes `surface: 'design'` on the flag in the catalog, and the control draws itself — a row of choices if your flag has `variants`, a plain on/off switch if it has none. Two optional fields narrow where it appears:

FIELD	LEAVE IT OFF AND	NAME IT WHEN
`versions`	the control follows you to every dashboard version you own — right for one exploration.	you have two versions and the control belongs to only one of them.
`surfaces`	it shows wherever `versions` already put it.	it only makes sense on some pages — `compass:course`, `compass:study-planner`. Added 16 October, after the Compass planner's controls turned up on a Home page that renders no Compass at all.

On your branch link the bar is right there, with a **Design** pill on the dark review bar to show and hide it. **Shift+D** hides both bars at once, and pressing it again with both hidden brings them back — the gesture to reach for just before a screenshot. It is ignored while you are typing into a field, so a capital D in the QA Notes composer is safe.

> Anyone opening your branch link can reach these controls, stakeholders included. A decision you want a reviewer to see *by default*, without going looking for it, belongs on the demo bar instead — which means asking Jill, because that bar is hers.

> What you never do: run the site on your own computer, edit `main`, or edit the Hub's own files. Claude will refuse and name them; ask Jill.

PART 4

Sharing on Refinement

Refinement is the board where work under discussion lives. Anyone adds to it, nothing on it is the source of truth, it filters by your name, and it has an archive.

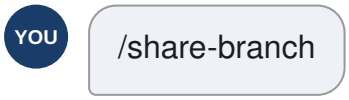
A row is one chunk of work, not a branch you live on. Since 15 October rows group under the **feature** they are chunks of, newest first. When a new chunk carries on from an older one, the older one folds under "1 earlier chunk" — still there, still

clickable — so a feature keeps a history you can open in a review. One long-running branch gives you a single link and no history at all.

> **Finish a chunk, share it, start a new branch for the next.**
>
> That is the habit the board is built around, and the reason small branches pay you back.

Adding your branch, the quick way

1. On your branch in the Code tab:

   It was called `/promote-to-refinement` until 15 October, and that name still works. It was briefly `/share` — but that name belongs to Claude Code itself, so the skill never loaded, which is why it is `/share-branch` now.

2. Claude asks once — checks, commit, push, open the form? — then runs the type check and the tests, commits anything uncommitted, pushes, waits for Netlify to build your branch, works out its address and a title, and asks what feature this is a chunk of. Nothing reaches the board or the To Do Board until you click the button on each of the two forms it opens.

3. In the form, **Added by** is a pick-list of the five of you. Pick your own name. The product (Atlas, Compass, Both) is already chosen from what your branch touched; change it if it is wrong.

   Feature is what this chunk is part of — start typing and the features already on the board are offered, so take one if it fits. **Replaces** appears only when that feature already has chunks, and defaults to your own newest one; its first option, *Nothing — these sit side by side*, is the right answer when two treatments should both stay up.

4. Click **Add link**. Your row appears at the top of its feature group, showing your name, when it was last saved, and the product.

5. **Then the card.**

   The To Do Board's card sheet opens next, also filled in, in the **Ready For Refinement** column — the row is the thing to look at, the card is who is waiting on it. Click **Add card**. If this chunk already has a card on the board, say so and Claude asks rather than opening a second one. Then it offers to cut your next branch.

Adding it by hand. Open the Hub, click Refinement, click Add link, paste the branch link from Part 3, type a title and a one-line note on what to look at, pick your name, pick the product, name the feature, Save.

Finding things. The pills above the list are the five names: All, Jill, Anjani, Eric, Afsana, Cody (the roster says **Jill**, which is what she signs). Click a name to see only that person's rows; click it again to clear. There is no product pill any more; the product is the tag on the row. Rows sit under their feature heading, so looking for a piece of work means looking for its name rather than its branch.

When a row's discussion is over

YOU WANT TO	DO THIS	WHAT HAPPENS
Tidy it off the board but keep it	Click the row's ⋮, choose **Hide**	The row disappears from the board. It is stamped with when and by whom. Nothing is deleted.
See what was hidden	Click **Show hidden · N** at the top right	The board switches to the hidden rows. **Back to board** returns.
Bring one back	In the hidden list, click its ⋮, choose **Unhide**	It is back on the board. Anyone can do this.
Delete a row that should never have existed	Click ⋮, choose **Remove**	Gone for good, after a confirmation.

Use Hide by default. Remove is for mistakes. (It was called Archive until 1 November; that word now means only the Archive section under Dev Handoff — things removed from the project.)

> **Two things to know.** The Updated tag changes only when the row is saved, not when your branch changes; after a big push, open the row and Save it again so the team sees fresh activity. And a Refinement row follows main underneath: if a component you point at changes on main, the row changes with it. Anything that must stay frozen, a user test for instance, is Jill's `/freeze-for-testing`, not a Refinement row.

PART 5

Making your surface the prototype, and who releases it

Prototypes is the one section that means "this is the product". It holds one row per surface, and each surface has an owner who decides what it should be. Since 1 November the section has a **Main** · **Stage** switch above the rows: the same five rows, and the switch picks which build each one opens. **Main** is the released product. **Stage** is what owners have decided and shared, waiting on Jill's release.

SURFACE ON PROTOTYPES	OWNER	REVIEW ADDRESS
End-to-End User Experience	Jill	held back — the row is a placeholder, but the build is still at `/dashboard-rebrand?demo=1`
Invite & Purchase	Afsana and Eric	`/end-to-end-demo?demo=1`
Atlas Learning	Eric	`/dashboard-rebrand?demo=1&section=dashboard`
Compass Learning	Anjani	`/dashboard-rebrand?demo=1&section=compass`
Admin Tool	Cody	placeholder until it is built

A chunk of work lives in three places, in order, and two different people move it:

PLACE	WHAT IT MEANS	WHO PUTS IT THERE	HOW
Refinement link	Your branch link. One chunk, under discussion.	You	`/share-branch` (Part 4)
Stage	The `stage` branch's build. Your surface as you have decided it should be, open to everyone from Prototypes → Stage, not yet released.	The surface's owner, or Jill	`/share-to-stage` (below)
Main	The released product. Prototypes → Main.	Jill only	`/release`

So there are two separate decisions, made by two different people:

- **What the surface should be** is the owner's call. The owner records it by promoting on their own branch and shares it to stage, where everyone can open it.
- **When stage becomes main** is Jill's call, and only she can do it. A release takes everything on stage at once — which is fine, because each of you owns one surface and stays in it. `/share-to-stage` refuses a branch that crosses into another owner's files.

When your branch is what your surface should become

1. Make sure the branch is current (Part 2) and the branch link looks right. You must own the surface the branch changes — Compass is Anjani's, Atlas Learning is Eric's, Invite & Purchase is Afsana and Eric's.

2. On your branch:

   YOU /share-to-stage

   It says where you are and what it is about to do, and asks once. Say yes.

3. **The flag questions.** If your branch changed any flag defaults and you have not yet decided them, it lists each one and asks: should this be the default, and which variant? Say yes to the ones that are your design and no to anything experimental you are not ready to stand behind. It writes the answers as a commit on your branch. (Behind the scenes this is the `decide-defaults` routine, which used to be called `promote-to-prototype`; you never need to type it.)

4. **Stage comes into your branch.** It merges today's `stage` into your branch so the two agree. If someone else's work touched the same lines, it stops and shows both sides — the same choice as a rebase conflict (Part 2): keep mine, keep stage's, keep both. If you cannot tell, say "Abort the merge" and ask the other owner; a conflict on stage is two owners in one file.

5. It runs the build and the tests, pushes your branch, and moves `stage` to it. Then it waits for Netlify to build the stage address and hands you the link.

6. On the Hub, **Prototypes** → **Stage** now opens your work for your surface. **Main does not**, yet. It also opens a card for the To Do Board's **On Stage** column, dated; click Add card. That column is the one place that says what is waiting on a release and since when.

7. Tell Jill, in words: "My Compass branch is on stage." Include the stage link.

8. Jill runs `/release`, which puts stage onto main. A few minutes later your surface's row under **Main** is your work, and its Updated tag changes to that moment. Main and Stage are the same build again until the next share.

9. Hide your Refinement row (Part 4), since the board is for things under discussion and this one is decided. Move your On Stage card along, or off the board.

10. Start the next piece from a fresh branch off main (Part 3, step 1). Do not keep working on the released branch.

If you own the surface, you can also decide something should NOT go in. Say so, archive the Refinement row, and tell Jill; she has a routine that records why a branch was tried and declined.

Stage is not a second Refinement. A Refinement link is one idea, isolated, that may change tomorrow; stage is a decision. Share to stage when you would be happy for Jill to release it that afternoon without asking you anything.

PART 6

Writing the dev handoff

A handoff is the page a developer builds a feature from. You write it with `/dev-handoff-notes`, in your own words. It becomes your own row on the Development board, in a file that is yours, with a status you set as the work moves. The only things you cannot do are release to main, which is Jill's, and share to stage for a surface you do not own (Part 5).

What it is. One page per feature at `/prototype/‹feature›`: one scroll, with a chip strip that jumps between its sections — **States**, **Options**, **CTAs**, **Questions**, **Edge cases**, **Design Spec**, and **Designer's Notes** last — and an **Open live preview** button in the header. A section you have not written yet still shows, with a note naming the gap, so a reader can tell "not written" from "there is none". The page exports as Markdown from its Actions menu — Share Link, Download .md, Copy .md — so a ticket can carry it.

> A developer will build exactly what the page says.
>
> Every line is a promise. A state you left out is a state they will not build; a line describing something you have since changed is something they will build wrong.

Have these ready before you start

The routine asks for them one at a time, component by component. It does not invent any of them, so a thin answer gives a thin page.

IT ASKS FOR	WHICH MEANS	THE SHAPE OF A GOOD ANSWER
The components	What the feature is made of. One is fine; six is fine. Each needs a name, where it lives on the page, and two lines on what it is for.	"Course Progress card — Home, top of the right column. Shows how far through the course the learner is and what to do next."
The states	Every version of each component, with *when* it appears and, where it is not obvious, *why*. This is the one list of states; there is no second one anywhere else on the page.	"No end date — when the learner has no access-expiry date. The Expires row does not render at all."
The buttons	Every control on the component and what it does, one line each, including the ones that are drawn and do nothing yet.	"Course Overview — opens the Course Overview page in Compass."
The edge cases	Asked and answered: a question and its answer. A developer arrives holding the question, so phrase it as one.	"Q: What if there is no image? A: The other parts of the card shift left."
The spec	The tokens it uses, how it behaves at each width, and where the design lives (the Figma link).	"Padding is space-4; the title is text-lg on neutral-900; under 768px the two columns stack."
The decisions	Each question that came up during Refinement, with the answer *and the reasoning*, marked Decided, Needs weigh-in or Blocked. The reasoning is the part that is still useful in six months.	"Flat list, because month headings fight a column sort."

IT ASKS FOR	WHICH MEANS	THE SHAPE OF A GOOD ANSWER
The switches	Which feature flag drives it, which brands it should be demoable in, and whether a member and a non-member both reach it.	"Flag `study-streak-v2`; both tiers; Atlas only."
Your notes	Asked last: anything engineering should know that the sections above have no slot for — sequencing, a caution, a preference. Blank unless you add something, and the page says so rather than hiding the section.	"Ship the card before the streak history endpoint exists; mock the count at 0 and we will wire it in the next sprint."

Running it

1 **On your branch, once the feature is built and the Refinement discussion has settled.**

YOU /dev-handoff-notes for the Study Streak card

Name the feature. If a handoff for it already exists, say "extend it" — the routine adds to the page rather than starting a second one. It first checks your name for the designer tag — the row says who wrote the handoff.

2 **Correct what it found, then answer six questions.**

It reads your component first and shows you what the code already says — the components, the states, the buttons — so most of your job is "did it miss one?". Then it asks, one at a time and in plain words: why a state exists where that is not obvious, the edge cases (no data, loading, failure, long text), what you decided and why, what is still open, and last, "anything else?" — that is your Designer's Notes, and "no" is a fine answer. Say "not decided yet" when that is the truth: it becomes a Needs weigh-in item a developer can see, rather than a gap they will fill in themselves.

3 **Let it check your words against the code.**

It reads the component you actually built and names where the two disagree. Fix the words, or the code, before going on. A handoff describing last week's version is worse than none.

4 **It writes your row.**

The handoff is one file, named after the feature, in `src/data/handoffs/`. It is yours: nobody else edits it, and Claude will change it for you whenever you ask. The routine runs the type check and the tests before it hands back, and a file that is wrong in a way the board cannot show fails those with the file named.

5 **Say yes.**

It ends with one question, "Commit and push?". Yes, and it hands you the link to your page on your branch link. The row is under Development there, next to the work it describes, and merges with that work.

6 **Move your card.**

On the To Do Board, drag this chunk's card to **Ready for Prototype and Handoff**. A card in that column is Jill's whoever put it there, which is how she knows there is a handoff to read. If the chunk has no card, add one.

7 **Set the status.**

A new handoff lands as **Not Ready**, so a page nobody has looked at never tells engineering to start. From there the status is yours. The one the team sees is in the file, so change it the way you change anything else on your branch: say "mark the Study Streak handoff Ready for Dev" to Claude, then push. The five are Not Ready, In Design, Needs Discussion, Blocked and Ready for Dev. The row's ⋮ on the Hub offers the same five, but that changes it in your own browser only — useful for trying a view, not for telling anyone.

8 **Read it the way a developer would.**

Open `/prototype/‹feature›` on your branch link and use find-in-page for a token name — the page is one scroll, so everything is there. Every section has its own address; copy it with the section in it when you want someone to read one thing.

> **Why one file each.** Until 26 October the row lived in the project list, which is one of the Hub's own files, so the routine was one you could start and Jill had to finish. Now a handoff is its own file and the project list is never touched. Two designers writing two handoffs never edit the same file, which is the same reason your components get a sibling copy (Part 3).

> Changed a component after its handoff was written? Run the routine again on the same feature. The page says what the build does, and a developer reading an old page builds an old thing.

PART 7

Reading the Hub

The Hub is at `ux-design-xceldashboard.netlify.app`, behind one password that Jill gives out. Everyone, stakeholders included, sees the same thing. Nothing inside is locked. Twelve sections in four groups:

GROUP	SECTION	WHAT IT IS FOR	WHO ADDS TO IT
Demo	**Prototypes**	The product as it stands: one row per surface, with a Main · Stage switch — Main is the released build, Stage is what owners have shared and Jill has not yet released.	Owners share to stage; Jill releases.
Design & Research	**Refinement**	Chunks of work under discussion, grouped by the feature they belong to. As many as you like. Filter by name, archive when done.	Anyone, on the page.
Design & Research	**Research**	Research links and rationale.	Anyone, on the page.
Design & Research	**Resources**	Briefs, Figma files, references. Things that live elsewhere — including this guide.	Anyone, on the page.
Design & Research	**PRDs**	The product requirements the designs answer.	Anyone, on the page.
Design & Research	**Explorations**	Standalone HTML documents: outside products and ideas rebuilt on our tokens. Five today.	Jill adds the row; send her the file.

GROUP	SECTION	WHAT IT IS FOR	WHO ADDS TO IT
Dev Handoff	**Development**	Handoff notes for engineering, each with a status chip.	You, with `/dev-handoff-notes` (Part 6): the row is your own file, and so is its status.
Dev Handoff	**Done**	Finished and signed off.	Jill.
Dev Handoff	**Archive**	Components and sections removed from the project, with how to bring each back.	Jill.
Dev Handoff	**QA Notes**	Design-versus-build findings with captures.	Anyone, on the page.
Team	**To Do Board**	Who is working on what, as five columns a card moves across — On Stage is the one that says what is waiting on a release.	Anyone, on the page.
Team	**Hub Resources**	The Hub's own guides — this one, Contributing, and the stakeholder page. Each opens in a new tab with a PDF beside it.	Code, not a board: the list changes when a file is added.

What went away on 15 October, in case you look for it: the second "demo" website, the Public toggle on Refinement rows, the locked Designers group in the nav, the Design section, and the Atlas / Compass filter pills on Refinement. The old Exploration section is the Explorations row under PRDs.

PART 8

When something goes wrong

None of these lose work. Find the message, read what it means, say the sentence.

YOU SEE	IT MEANS	SAY TO CLAUDE
"BLOCKED: that pushes to main" (or "moves you onto main")	You asked for something that would change main, which is Jill's. Claude stopped before running it; GitHub would have refused it too.	"Switch back to my feature branch." Then `/share-branch` to put the work in front of the team, and `/share-to-stage` when your surface is decided; Jill releases stage to main.
A refusal naming a file, like "UxDashboardPage.tsx is protected"	That file is the Hub's own machinery. Only Jill edits it.	Nothing. Tell Jill what you needed changed.
"Updates were rejected because the tip of your current branch is behind"	You rebased, so your branch's history changed, and a normal push is refused.	"Push with force-with-lease." Your branch only. Never main.
"CONFLICT (content): Merge conflict in …"	Main and your commit changed the same lines. The rebase paused for you to choose.	"Show me both sides." Then "Keep my version" / "Keep main's version" / "Keep both", then "Continue the rebase." Lost? "Abort the rebase" and ask Jill.
"fatal: 'main' is already checked out at …"	Someone tried to switch to main in a folder where that is not allowed. You should not be switching to main anyway.	"Switch back to my feature branch."

YOU SEE	IT MEANS	SAY TO CLAUDE
The build fails with lines starting `error TS`	A type error. Netlify would refuse to deploy this.	"Fix the type error and run the build again." If it is in a file you never touched, stop and ask Jill.
Tests fail, and the names mention `TestingGateway` on the first run only	A known slow first run when caches are cold.	"Run the tests again." If it fails twice, it is real.
The Netlify link asks for a password	Every branch link is behind the same password as the Hub.	Nothing. Enter the Hub password.
The Netlify link shows an old version, or "Page not found"	The build has not finished, or failed.	"Check the Netlify deploy status for my branch." If it failed, "Show me why"; usually the type error above. Or look for yourself on the deploys page.
"You are in 'detached HEAD' state"	Your work has no branch under it. Nothing is lost yet.	"Create a branch from here called feat/‹name› and switch to it."
Claude says it cannot run a command or lacks permission	The permission card was denied, or needs approving.	Click Allow on the card, or "Try again."
Your Refinement row shows an old Updated time	The stamp changes when the row is saved, not when the branch changes.	Open the row on the Hub, click Save.
Your commits show the author as "Claude"	Your git name was never set on this computer.	Set it, per the Contributing guide. Then: "Re-author my commits on this branch to me."

YOU SEE	IT MEANS	SAY TO CLAUDE
While writing a handoff, a refusal naming `prototypeFeatures.ts`	Claude tried to put the row in the project list instead of its own file.	"Put the handoff in its own file under src/data/handoffs/, named after the feature." Then carry on (Part 6).
Your design control shows up on a page that has nothing to do with it	The flag has no `surfaces`, so it appears everywhere the version it belongs to is on screen.	"Scope my design control to the Compass surfaces only."
You are not sure what Claude just did	Anything.	"Summarise what you changed in the last step, file by file." Then decide.

> If none of these fit, do not try a second fix on top of the first. Copy the message, send it to Jill, and leave the folder as it is.

PART 9

The routines, and when to use each

A routine is a checklist Claude runs with you. Type `/` and its name in the chat box. It asks you a few questions, does the steps in order, and stops short of anything it should not decide for you.

Four of them are yours. Four belong to the design lead (Jill); know their names so you recognise them when she mentions one. The word is **routine** — the files call them skills and Claude calls them commands; same thing.

Which one do I want?

YOU ARE THINKING	REACH FOR
"I want eyes on this one card, not the whole branch."	`/share-component`
"I want the team to look at my branch."	`/share-branch`
"This is what my surface should be."	`/share-to-stage` — it asks the flag questions, puts your surface on the stage build, and files the On Stage card. Then tell Jill.
"Release what is on stage."	Jill's — `/release`
"It's built and discussed. Engineering needs to read it."	`/dev-handoff-notes`, then move your card to Ready for Prototype and Handoff
"I'm done discussing this row."	Hide, on the row's ⋮ (no routine)
"We tried it and decided not to."	Tell Jill; she runs `/retire-from-refinement`
"I need a frozen link for a user test."	Tell Jill; she runs `/freeze-for-testing`

Yours

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NE DO
`/share-component`	One component on your branch wants feedback and the rest of the branch would distract from it.	Which component (name it, or select it in the browser pane first). Which of its flag's variants to show side by side. A note telling the reviewer what to look at.	Finds the feature flag that drives the component. If there is none, shows you the flag it would add (its name, label, the one line it changes in your component) and waits for your yes. Checks the branch is pushed and built. Builds one review page with every variant stacked, each pinned. Opens Refinement's Add link form with it filled in.	Sa the (yo cli Ad Ad fla wi sh yo firs Sh the wh bra

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NE DO
`/share-branch`	A chunk of work is ready for the team to look at.	One yes, up front, covering the checks, the commit and the push. A title. A one-line note on what to look at. What feature this is a chunk of. Which product (it guesses from the files you touched: Compass files mean Compass).	Checks you are on a branch, not main. Says so if the branch has got long. Runs the type check and the tests. Commits anything uncommitted and pushes. Works out the Netlify address and waits for the build to finish. Opens Refinement's Add link form with everything filled in, your name included. Then opens the To Do Board's card sheet, filled in, in **Ready For Refinement**. Offers to cut your next branch.	Sa the or ca (yo cli **Ac lin** the **Ac ca** Ru ma Co or wit the it a yo

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NE DO
`/dev-handoff-notes`	The feature is built, the Refinement discussion has settled, and engineering needs to read it. Also when a component changes after its handoff was written.	Component by component: its states with when and why, every button and what it does, the edge cases as question and answer, the tokens and responsive behaviour, the decisions with their reasoning, the flag, the brands, member and non-member.	Reads the component you built and names where your words and the code disagree. Writes the row as its own file under `src/data/handoffs/`, named after the feature, status Not Ready, and runs the checks. Tells you what is still unwritten.	Inv sta a rea yo no giv To the pro list Sh ma pro a su yo no ow

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NE DO
`/share-to-stage`	Your branch is what your surface should become, and you own that surface. The step after Refinement, before Jill's release.	One yes, up front, covering the merge, the checks and the pushes. The `/promote-to-prototype` flag questions, if your branch has not answered them yet.	Checks every file you changed belongs to a surface you own. Runs the flag questions if needed. Merges today's stage into your branch (stops on a conflict and shows both sides). Runs the build and the tests. Pushes your branch and moves stage to it. Waits for the stage build and hands you the link. Opens the On Stage card filled in.	To ma Me a su yo no ow Fo pu sta Re a co for

The design lead's: recognise, don't run

ROUTINE	WHAT IT IS FOR
`/release`	Merging `stage` into main — everything on stage, at once. The only way anything reaches the product. It refuses a stage with an undecided flag group. Since 29 October it is enforced for everyone else: Claude refuses the commands before they run, and GitHub refuses the push even if something got past Claude. (`/ship-to-main` is the old name and still answers, but a branch no longer goes to main directly — it goes to stage first.)
`/freeze-for-testing`	Freezing a branch onto the separate user-test site and handing back participant and moderator links. A frozen link does not move when main does.
`/retire-from-refinement`	Closing a Refinement row properly when the work was released or declined: deciding what happens to the branch and writing the record of why. Different from Hide on the kebab, which only hides.
`/archive-a-feature`	Retiring a component or variant without deleting it, and writing the Archive-section row that says how to bring it back.

PART 10

Scheduled routines

Claude Code can run a job on a schedule: you describe it once and it runs in your project folder, in a fresh session, and sends you a note when it is done. Two are worth having — a rebase every weekday morning, and a sweep for old backup branches on Fridays.

Two things to know about either of them:

- **It runs while the Claude app is open.** If your laptop was closed at 8:30, it runs the next time you open the app. In practice that means "when I start my day".
- **It works on whichever branch is checked out** in your `xcel-ux-dashboard` folder. That is the branch you were on when you closed the app yesterday, which is almost always the one you want.

The morning rebase

It makes Part 2 something that happens while you get coffee. Open the Code tab on the project and paste this, exactly:

PASTE TO CLAUDE

Create a scheduled task called morning-rebase that runs every weekday at 8:30 in this project folder. Each run, do the following and nothing else:

1. Report the current branch. If it is `main`, or there is no branch checked out, stop and say so.
2. If there are uncommitted changes, stop and say so. Do not stash them and do not commit them.
3. Fetch origin. If the branch is zero commits behind origin/main, report "already current" and stop.
4. Create a local backup branch named backup/‹branch›/‹today's date› pointing at the current commit. Do not push it.
5. Rebase the branch onto origin/main. If any file conflicts, abort the rebase so the branch is exactly as it was, and report which files conflicted and in which commit.
6. If the rebase succeeded, run `npm run build` and then `npx vitest run`. If either fails, do not push. Report what failed; say that the local branch is rebased but origin still has yesterday's version.
7. If both passed, push the branch to origin with `--force-with-lease`. Never push main. Never push without the lease.
8. Finish with a six-line summary: the branch; how many commits behind it was; whether there were conflicts; whether build and tests passed; whether it pushed; and the Netlify branch link for the branch.

Never run `git switch main`, never edit files, never resolve a conflict yourself, and never delete anything.

Claude confirms the schedule and the task name. Say yes.

What you see each morning

A short note in the Claude app: rebased and pushed, or stopped and why. If it says it stopped on a conflict, do it by hand — Part 2, "If it stops on a conflict". If it says checks failed, open the Code tab and say:

YOU

> Show me what failed in the morning rebase.

Useful sentences

YOU WANT TO	SAY
See what is scheduled	"List my scheduled tasks."
Run it right now instead of waiting	"Run the morning-rebase task now."
Change the time	"Change morning-rebase to run at 7:45 on weekdays."
Pause it for a week	"Disable morning-rebase until next Monday."
Clean up the backups it made	"Delete my local backup/ branches older than two weeks." Or set up the sweep below.

> It never touches main, never writes code, and never pushes anything that did not build and pass. The worst morning it can give you is "stopped, here is why".

The Friday backup sweep

Every rebase leaves a `backup/…` branch behind — the morning task makes one too. They are cheap to keep and easy to forget, and after a month there are twenty. This one finds the ones that have done their job. It does not delete anything.

PASTE TO CLAUDE

Create a scheduled task called backup-sweep that runs every Friday at 16:00 in this project folder. Each run, do the following and nothing else:

1. List every local branch whose name starts with `backup/`, and every branch on origin whose name starts with `backup/`. Match on the prefix only — the rest of the name is not a fixed shape.
2. For each one report: its name, how many days old its newest commit is, the branch it was made from, and whether that branch still exists locally and on origin.
3. Sort them into two lists. Delete nothing.
    - **Safe to delete** — older than fourteen days, AND the branch it was made from still exists, AND that branch has no commits that are missing from origin.
    - **Keep** — everything else, with the reason on each: too new, the branch it backed up is gone, or that branch has work that has not been pushed.
4. If a backup's source branch no longer exists anywhere, say so loudly and put it in Keep: that backup may be the only copy of the work.
5. Finish with the exact command that would delete the safe ones, local and remote listed separately, and stop there. Never delete a branch, local or on origin, unless I say yes in that conversation.

Never touch a branch that is not named `backup/…`, never push, and never run `git switch`.

Read the two lists, and if you agree, say so:

YOU

Delete the backups you listed as safe, local and remote.

> **Why it asks rather than decides.** A rebase rewrites your commits, so a backup is never "merged" in git's sense and `git branch -d` will refuse it — deleting one needs the force form, `-D`. That is exactly what the backup was for, which is also why nothing here deletes on your behalf. If you are ever unsure about one, keep it: a stale branch costs nothing.

PART 11

Glossary

TERM	MEANING HERE
Claude Code	The Code tab in the Claude desktop app. A chat the files in a folder, and run the project's checks f
Repository, repo	The project's whole history and files, on GitHub a computer. Ours is `jillmoreno/xcel-ux-dashboa`
Clone	Copying the repository to your computer the first
Design lead	Jill Moreno. The person in charge of the project's she owns `main`, runs `/release`, and sets the d "Ask Jill" throughout these guides means her.
main	The branch that is the product. The design lead's
Branch	Your own line of work, named `feat/…`. Netlify bu own address.
Commit	A saved snapshot with a message. Several per d
Push	Sending commits to GitHub. Netlify builds after e
Fetch	Asking GitHub what has changed, without chang
Rebase	Replaying your commits on top of today's main. D as often as you like.
Force push with lease	The push you need after a rebase. Safe on your someone else pushed there first. Never on main.
Conflict	Main and your commit changed the same lines. Y
Release (was ship to main)	Jill putting `stage` into main with `/release`. The reaches main. Everything on stage goes at once.
Stage	The `stage` branch and its build, between a Refi what owners have decided and shared, waiting o Prototypes → Stage.

TERM	MEANING HERE
Share to stage	The owner putting their surface onto stage with `/` the flag questions if the branch has not answered
Main · Stage	The switch above the Prototypes rows. Same row released build, Stage opens the stage build.
On Stage	The To Do Board column for a chunk that is on st release, with the date in its note.
Feature flag	A named switch in the code that picks between th something. On by default on your branch; Jill dec
Design controls bar	The light cyan strip carrying the decisions still op You add to it by marking your flag `surface: 'de` the bar. The amber bar beside it is the demo bar,
Sibling copy	Copying a component to a V2 file instead of editi edit the same file. `/share-component` does this.
Scope, owner	Each Prototypes row is a scope with an owner wh be: Compass is Anjani's, Atlas Learning is Eric's, Afsana's and Eric's, Admin is Cody's, End-to-End
Chunk	One piece of work, one branch, one Refinement feature group together on the board.
Feature (on Refinement)	What a chunk is part of, and the heading its row s the board's existing features offered as you type.
Share (was promote to refinement)	Putting one chunk of your branch's work on the P discussion. Anyone, any time.
Decide defaults, the flag questions	The owner recording which flags are their surface branch. Not a release. A step inside `/share-to-` it was called `promote-to-prototype`.

TERM	MEANING HERE
Branch link (was "review link", "branch build")	Your branch as a website: `https://‹branch›--ux-design-xceldashboard`
`?demo=1`	Turns on the demo controls bar, so a reviewer ca progress and tier.
Hide (Refinement)	Hiding a row from the board, reversibly, with a sta Was called Archive until 1 November.
Archive	The nav section listing things removed from the p each back. The only thing called Archive now.
Explorations	Standalone HTML documents on the Hub. Not th
Dev handoff	The page a developer builds a feature from — co buttons, edge cases, spec and decisions — at `/` You write it with `/dev-handoff-notes`, and the r
Development (section)	Where handoffs wait for engineering, each with a Handoff in the nav.
Not Ready	The status every handoff lands with, so an unrea engineering to start. Yours to change, in the file. for your browser only.
Pull request, PR	GitHub's page for a branch that is proposed for m one: your branch goes to stage, and Jill releases appears in GitHub's own messages.